Persistent In-Memory Execution for CodeIgniter 4: Impact of the FrankenPHP Worker Mode Compared with PHP-FPM and Laravel Octane

Anonymous Author(s)

ABSTRACT

Persistent in-memory execution removes the per-request framework bootstrap of traditional PHP, but CodeIgniter 4 only gained native support for the FrankenPHP Worker mode in version 4.7. This paper measures, under the same resource budget (1.0 CPU, 512 MB), the capacity, latency and CPU/memory use of CodeIgniter 4 under PHP-FPM, classic FrankenPHP and FrankenPHP Worker mode, with Laravel Octane as a reference, all on the same Debian 13 base image, using k6, Prometheus and Docker. Throughput was measured under a closed-loop workload with 100 virtual users (five 15-minute runs per scenario, 10-minute plateau) and latency below saturation in an open-loop test at a fixed 100 requests per second (three runs). At the plateau, mean throughput was 143.6 requests per second (PHP-FPM), 155.7 (classic FrankenPHP), 304.4 (CodeIgniter 4 Worker) and 337.8 (Laravel Octane), with no errors. Worker mode raised CodeIgniter 4 throughput by 112.1% over PHP-FPM (95.5% over classic FrankenPHP) and cut p95 latency at 100 requests per second by 57% (15.3 to 6.6 ms), at the cost of about 46 MiB of additional memory; memory stayed stable in 30-minute tests. The PHP-FPM comparison involves different server architectures, and the gain combines persistent execution and database connection reuse, which the experiment does not separate, although per-endpoint data suggest the connection dominates the latency reduction. Classic mode brought a modest throughput gain (8.5%), but no measurable latency improvement below saturation. In this environment, Worker mode pays off for CodeIgniter 4, approaching Laravel Octane.

KEYWORDS

CodeIgniter 4, FrankenPHP, Worker mode, performance benchmarking, PHP

1 INTRODUÇÃO

O PHP tradicionalmente executa cada requisição HTTP de forma isolada: o código é carregado, executado e os recursos são liberados ao final. O PHP-FPM mantém um *pool* de processos, mas a inicialização do *framework* (carga de classes, serviços e configuração) se repete a cada acesso, o que limita a vazão em cenários de alta carga. Servidores de aplicação com memória persistente, como o FrankenPHP, mantêm a aplicação carregada e eliminam essa repetição [6] e, por serem integrados ao servidor web, também a comunicação entre processos do modelo com PHP-FPM [5]. O Laravel Octane consolidou essa prática no ecossistema Laravel [13], e o CodeIgniter 4 (CI4) passou a oferecer suporte nativo ao modo Worker apenas a partir da versão 4.7 [4].

Para quem mantém aplicações em CI4, a pergunta prática é se o modo Worker justifica a adoção ou se trocar apenas o servidor, mantendo o modo clássico, já bastaria. Isso importa para a eficiência de recursos em nuvem e para a modernização de sistemas legados sem reescrita. Este trabalho mede, sob o mesmo orçamento de recursos, a vazão observada sob carga fechada de 100 usuários virtuais, a latência sem saturação e o consumo de CPU e memória do CI4 sob PHP-FPM, FrankenPHP clássico e FrankenPHP em modo Worker, usando o Laravel Octane como referência, e verifica o que cada arquitetura preserva entre requisições.

2 FUNDAMENTAÇÃO TEÓRICA

No modelo tradicional, o PHP não preserva estado entre requisições. O trabalho de conclusão de curso de Luz [12] compara o modelo síncrono do PHP com o assíncrono do Swoole em testes de carga com ANOVA e teste t, e conclui que o modo de operação é o fator determinante do desempenho. O FrankenPHP, construído sobre o servidor Caddy, tem um modo clássico, que substitui o PHP-FPM com um número fixo de *threads* (por padrão, o dobro dos núcleos de CPU visíveis), e um modo Worker, em que o *script* de entrada é carregado uma vez e atende requisições em laço, ao custo de o programador evitar vazamento de estado entre elas [6]. Azzahidi et al. [2] avaliaram *frameworks* de API REST com k6 e destacaram o Laravel com FrankenPHP pelo uso eficiente de memória. Dhaniswara et al. [5] compararam, para uma aplicação Laravel 12 em contêineres Docker com limites idênticos de recursos, o PHP-FPM com Nginx e o FrankenPHP em modo Worker com Octane, usando o Apache JMeter (100 usuários virtuais, 10 minutos). Em um *endpoint* de carga leve, o FrankenPHP teve tempo de resposta médio 15,8% menor e vazão 18,6% maior, com mais memória; em um *endpoint* que serializa grande volume de dados, as duas arquiteturas saturaram a CPU, e os autores atribuem o gargalo à aplicação.

O CodeIgniter 4 é um *framework* leve, e o suporte nativo ao modo Worker a partir da versão 4.7 introduziu um *script* de entrada próprio, gerado pelo comando `php spark worker:install` [4]. Já o Laravel Octane mantém o *framework* inicializado e trata a limpeza do estado entre requisições [13]. O estudo mais próximo [5] avalia o Laravel e confronta o modo Worker diretamente com o PHP-FPM; os demais comparam modelos de execução ou *frameworks* distintos. Nas buscas que realizamos, não encontramos estudo revisado por pares que inclua o modo clássico do FrankenPHP como cenário intermediário, para separar o efeito do servidor do efeito do modo Worker, nem que o faça no CodeIgniter 4, que é a lacuna abordada aqui.

Quanto ao rigor, *benchmarks* de *software* são instáveis e exigem repetição e controle do ambiente [10], e comparações dependem de condições idênticas e reprodutíveis [3, 9, 15]. Há ainda uma escolha de modelo de carga. No modelo fechado, um número fixo de usuários só dispara uma nova requisição após a resposta anterior; o servidor lento reduz a própria carga, e o comportamento

difere muito do modelo aberto, em que as chegadas independem das respostas [8, 14]. Em sistemas estáveis, a Lei de Little [11] relaciona o número médio de requisições no sistema, a vazão e o tempo médio de permanência ($N = XW$). Num teste fechado, a latência pode ser dominada pela fila quando o servidor se aproxima da saturação ($W \approx N/X$), mas a lei não separa espera de tempo de serviço nem explica percentis. Por isso este trabalho usa o modelo fechado para medir a *vazão observada com 100 usuários virtuais* e um teste de taxa de chegada fixa, abaixo da vazão observada em todos os cenários, para medir a *latência*.

3 MATERIAIS E MÉTODOS

O estudo é experimental e quantitativo. Foram avaliados quatro cenários, cada um limitado a 1,0 CPU e 512 MB de RAM: (A) CI4 4.7.5 com PHP-FPM (4 processos estáticos) e Nginx 1.31.7, com o orçamento dividido em 0,85 CPU/448 MB e 0,15 CPU/64 MB; (B) o mesmo aplicativo no FrankenPHP 1.13.0 em modo clássico, com 4 *threads*; (C) o mesmo aplicativo no FrankenPHP em modo Worker, com 4 *workers* e o suporte nativo do CI4 4.7; e (D) Laravel 13.36 com Octane 2.22 sobre FrankenPHP, com 4 *workers*. Um cenário complementar de sensibilidade (E) repete o B sem a diretiva `num_threads`, de modo que o FrankenPHP usa seu número padrão de *threads*, o dobro dos núcleos de CPU visíveis; com o limite de 1,0 CPU, isso resultou em 2 *threads* (contra 4 no B), valor confirmado no registro de inicialização do servidor. Todas as imagens de aplicação e servidor web usam a mesma base (Debian 13) e o mesmo PHP 8.3.35, com versões e *digests* fixados; a única diferença de compilação, inerente às arquiteturas, é o PHP *thread-safe* do FrankenPHP contra o não *thread-safe* do PHP-FPM. O OPcache usa a mesma configuração em todos os cenários e o banco é um PostgreSQL 16 idêntico (0,5 CPU, 256 MB), fora do orçamento dos cenários. Os *endpoints* têm o mesmo código nos cenários A, B, C e E (um controlador do CI4), e equivalente no Laravel do cenário D.

Table 1: Configuração dos cenários (todos com base Debian 13 e PHP 8.3.35).

Cen.	Aplicação	Servidor	Concorrência
A	CI4 4.7.5	Nginx + PHP-FPM	4 processos estáticos
B	CI4 4.7.5	FrankenPHP 1.13.0 clássico	4 *threads*
C	CI4 4.7.5	FrankenPHP, modo Worker	4 *workers*
D	Laravel 13.36 + Octane 2.22	FrankenPHP, modo Worker	4 *workers*
E	CI4 4.7.5	FrankenPHP 1.13.0 clássico	2 *threads* (padrão)

Os quatro *endpoints* são: `/health` (resposta mínima, sem banco), `/compute` (serialização JSON de 1.000 elementos), `/memory` (contador estático e PID) e `/db` (consulta por chave primária e PID da conexão PostgreSQL), na proporção 10/30/30/30%. A carga foi gerada pelo k6 2.3 em contêiner separado (2,0 CPU) [7], em dois testes. *(i) Vazão observada com 100 usuários virtuais*, em modelo fechado: 15 minutos por execução, com 2 min de aquecimento até 10 usuários virtuais, 2 min de rampa até 100, patamar de 10 minutos e descida de 1 min, sem espera entre requisições; $n = 5$ execuções por cenário (três em 19/out/2026, com A, B, C e D em ordem rotativa para reduzir viés de ordem e temperatura e o E em bloco ao final, e duas em 20/out/2026, com A–E em ordem rotativa, como também na taxa fixa). *(ii) Latência sem saturação*, em modelo aberto (`constant-arrival-rate`): 100 requisições por segundo, escolhidas por corresponderem a cerca de 70% da vazão observada no cenário A no teste fechado (aprox. 144 RPS), com 60 s de aquecimento e 8 minutos de patamar, três execuções por cenário; o teste só é válido se nenhuma iteração for descartada, o que se verificou em todas; a CPU abaixo de 60% indicou que a carga ficou abaixo da saturação. Em ambos os testes, RPS, percentis e latências médias são do patamar (aquecimento, rampa e descida excluídos; RPS é o número de requisições do patamar dividido pela sua duração), e CPU e RAM (`docker stats`) são médias da mesma janela. Antes de cada execução, o *cache* do sistema era limpo e o Prometheus reiniciado. Também se fez um diagnóstico de persistência por *worker* e um teste de 30 minutos de patamar para C e D. Tudo rodou em um notebook com AMD Ryzen 7 5700U e 14 GiB de RAM, com Docker 28.1.1, e os *scripts* reproduzem a coleta completa. O código e os dados brutos estão disponíveis [1].

Para cada cenário calcularam-se média e intervalo de confiança de 95% (IC, t de Student). As comparações par a par (seis por métrica: B–A, E–B, C–A, C–B, C–E e D–C) usam o teste t de Welch, que dispensa variâncias iguais, com IC da diferença e correção de Holm sobre essas seis; nas comparações principais, os valores de p reportados são os do teste de Welch ajustados por Holm. O IC95% da razão de médias foi calculado pelo método delta na escala logarítmica (t com graus de liberdade de Welch–Satterthwaite). A correção de Holm foi aplicada separadamente a cada métrica e modelo de carga, sobre as seis comparações acima: RPS, p95 e p99 no teste fechado e p95 no teste aberto; CPU, RAM e p50 são descritivos, e não houve outras comparações exploratórias. O Mann–Whitney exato é apresentado separadamente, sem ajuste para comparações múltiplas, como análise complementar; a conclusão principal repousa no Welch com Holm (com 5 contra 5 observações o menor p possível é 0,0079; com 3 contra 3, 0,1). Com n pequeno, a normalidade não pode ser verificada; os testes servem para comparar a variação entre execuções no mesmo ambiente, e não generalizam para outro *hardware*.

4 RESULTADOS

Em todas as execuções a taxa de falhas HTTP foi 0,00%. Foram realizadas cinco execuções por cenário em dois dias (três em 19/out e duas em 20/out), e todas entram na análise principal ($n = 5$). Como verificação descritiva, as médias de 20/out diferiram em menos de 3% das de 19/out em RPS e p95 em todos os cenários (B: +2,8% no p95).

4.1 Vazão observada com 100 usuários virtuais (modelo fechado)

A Tabela 2 e a Figura 1 resumem a vazão observada com 100 usuários virtuais, que não é a capacidade máxima de cada cenário, mas o valor medido nessa carga. O modo Worker do CI4 (C) atingiu 304,4 RPS contra 143,6 do PHP-FPM (A), ganho de 112,1% (razão 2,12; IC95% 2,10–2,14). O modo clássico trouxe ganho pequeno: +8,5% com 4 *threads* (B) e +22,9% com o número padrão, 2 *threads* (E). Comparado ao B, que usa o mesmo servidor e o mesmo número de unidades de concorrência, o C obteve +95,5%. O Laravel Octane (D) obteve 337,8 RPS, 11,0% acima de C. A Tabela 3 traz as comparações: após a correção de Holm, todas as diferenças de RPS têm

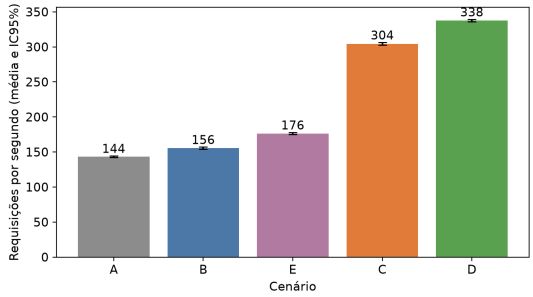


Figure 1: Vazão média por cenário em modelo fechado (patamar), com IC95% de 5 execuções.

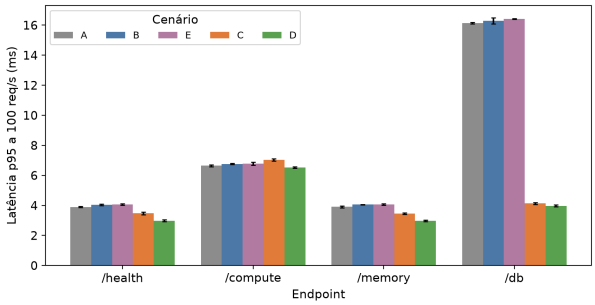


Figure 2: Latência p95 por *endpoint* a 100 req/s (barras: IC95% de 3 execuções).

$p < 0{,}001$ e as de p95, $p \leq 0{,}027$ (B contra A) ou menor; no p99, B não difere de A (−0,6%; $p = 0{,}50$). O Mann–Whitney, em separado, resulta em $p = 0{,}0079$ (o mínimo possível) em todas as comparações de RPS e p95, pois as cinco observações de cada cenário não se sobrepõem. A CPU ficou entre 92,7% (A) e 100,6% (D) de um núcleo, ou seja, o gargalo é a CPU, e nenhum cenário passou de 173 MiB de RAM.

Table 2: Vazão sob 100 usuários virtuais, patamar de 10 min, média de 5 execuções. IC95% (t de Student) apenas em RPS e p95; p99, CPU e RAM são médias.

Cenário	RPS	p95 (ms)	p99 (ms)	CPU (%)	RAM (MiB)
A – PHP-FPM	143,6 ± 1,1	788,9 ± 3,1	804,8	92,7	79,1
B – Clássico	155,7 ± 1,5	751,4 ± 30,6	800,1	100,1	123,7
E – Clássico, padrão	176,5 ± 1,6	679,5 ± 13,2	737,0	97,5	115,7
C – CI4 Worker	304,4 ± 2,1	397,6 ± 0,8	408,0	100,4	125,5
D – Octane	337,8 ± 1,7	383,1 ± 1,0	399,4	100,6	171,5

Table 3: Comparações em modelo fechado ($n = 5$; B–A indica B contra A): variação percentual, IC95% da diferença (Welch) e p do Welch ajustado por Holm (seis comparações por métrica).

	RPS			p95		
Par	Var. (%)	IC95%	p	Var. (%)	IC95% (ms)	p
B–A	+8,5	[+10,5; +13,8]	$< 0,001$	−4,8	[−68,0; −6,9]	0,027
E–B	+13,3	[+18,9; +22,6]	$< 0,001$	−9,6	[−102,1; −41,8]	0,003
C–A	+112,1	[+158,8; +162,9]	$< 0,001$	−49,6	[−394,4; −388,3]	$< 0,001$
C–B	+95,5	[+146,5; +150,9]	$< 0,001$	−47,1	[−384,5; −323,3]	$< 0,001$
C–E	+72,5	[+125,8; +130,2]	$< 0,001$	−41,5	[−295,2; −268,7]	$< 0,001$
D–C	+11,0	[+31,1; +35,6]	$< 0,001$	−3,6	[−15,5; −13,4]	$< 0,001$

4.2 Latência sem saturação (100 requisições por segundo)

No modelo fechado, o p95 de C é 49,6% menor que o de A (e o de B, 4,8% menor que o de A, $p = 0{,}027$), mas isso reflete a fila, já que o servidor está com a CPU no limite: como contexto, $N/X = 100/143{,}6 \approx 697$ ms (Lei de Little, que se refere à média e não ao p95) contra latência média medida de 696 ms em A (328 ms contra 328 ms em C). A Tabela 4 mostra a latência sem saturação. A 100 req/s todos os cenários sustentaram a taxa, sem descartes, e a CPU ficou abaixo de 60% (A, B, E) e de 36% (C, D). O p95 do modo Worker do CI4 é 6,6 ms contra 15,3 ms do PHP-FPM, 57% menor (IC95% da diferença −8,85 a −8,67 ms; Welch com Holm, $p < 0{,}001$) e 57% menor que o do B (15,4 ms; Welch com Holm, $p < 0{,}001$). O modo clássico não alterou a latência de forma mensurável abaixo da saturação: B e E diferem de A em menos de 0,2 ms (B contra A, $p = 0{,}079$; E contra B, $p = 0{,}092$, ambos ajustados por Holm). O Octane teve p95 6,2 ms, 6% abaixo de C ($p = 0{,}009$, Welch com Holm, $n = 3$).

Table 4: Latência a 100 req/s (modelo aberto, patamar de 8 min, média de 3 execuções): p95 com IC95%, p99, CPU e latência média (ms) dos quatro *endpoints*.

Cen.	p95	p99	CPU (%)	/health	/compute	/memory	/db
A	15,34 ± 0,08	16,5	55,9	3,2	5,8	3,2	14,6
B	15,44 ± 0,11	16,6	57,7	3,3	5,9	3,3	14,7
E	15,52 ± 0,02	16,8	57,9	3,3	5,9	3,3	14,8
C	6,58 ± 0,10	7,1	35,5	2,8	5,7	2,8	3,3
D	6,17 ± 0,02	6,7	36,1	2,4	5,3	2,4	3,3

A redução de latência concentra-se no `/db`: sua latência média cai de 14,6 ms (A) para 3,3 ms (C), ao passo que `/health`, que não acessa o banco, cai de 3,2 para 2,8 ms, `/memory`, de 3,2 para 2,8 ms e `/compute`, de 5,8 para 5,7 ms (Figura 2). A CPU para atender os mesmos 100 req/s também cai de 55,9% para 35,5%.

4.3 Estado preservado e estabilidade

O diagnóstico de persistência (Tabela 5) mostra que C e D preservam o estado e a conexão com o banco por *worker*, ao passo que A, B e E reiniciam ambos a cada requisição. Com um único *worker*, 200 requisições sequenciais a `/memory` produziram um contador crescente de 2 a 201 e um único PID de conexão em C e em D; com quatro *workers* e requisições concorrentes, o C usou quatro processos, cada um com seu contador e sua conexão. No D, o diagnóstico registrou um *worker* adicional, coerente com o reciclo periódico de processos do Octane (cerca de 500 requisições), que não ocorreu no CI4. No teste de 30 minutos (Figura 3), a RAM do C ficou em 125,8 MiB nos primeiros 5 minutos e em 126,0 MiB nos últimos, e a do D em 171,3 e 171,8 MiB; a vazão ficou 2,5% (C) e 1,3% (D) abaixo da das execuções de 15 minutos.

Table 5: Estado preservado entre requisições.

Cenário	Contador	Conexão com o banco
A, B, E	reinicia (sempre 1)	nova a cada requisição
C, D	cresce por *worker*	mantida por *worker*

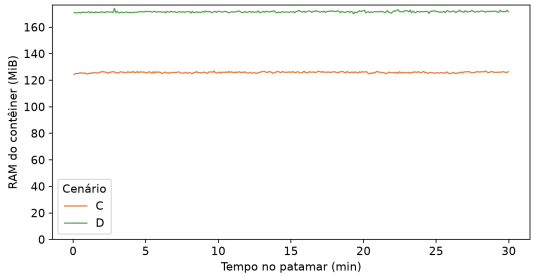


Figure 3: RAM dos contêineres C e D durante 30 minutos de patamar.

5 DISCUSSÃO

O resultado mais forte é que manter o processo e o *framework* em memória mais que dobrou a vazão do CI4 sob 100 usuários virtuais (+112,1%) e reduziu em 57% o p95 a 100 req/s, abaixo da saturação, com os mesmos recursos, o que é coerente com o princípio do modo Worker [6] e com a direção dos resultados de Luz [12] e Azzahidi et al. [2]. O cenário B mostra que trocar apenas o servidor rende pouco em vazão (+8,5%); no teste aberto, abaixo da saturação, não houve melhora mensurável de latência, e no fechado a pequena redução do p95 (4,8%) deve ser lida considerando a fila, e o E, com 2 *threads* (menos que as 4 do B), teve mais vazão que o B, o que indica que o modo clássico depende do número de *threads*; uma hipótese, não investigada, é a menor disputa pela cota de uma CPU já saturada. Mesmo assim, o E fica 42% abaixo do C em vazão. Há dois níveis de evidência. A comparação mais próxima do efeito do modo Worker é a de B com C, que usam o mesmo FrankenPHP e quatro unidades de concorrência (+95,5% de vazão e −57% de p95 sem saturação), embora ainda combine o modo de execução com o reaproveitamento de conexões. A de C com A compara duas arquiteturas completas (servidor, compilação do PHP e divisão de recursos diferentes): o ganho observado de 2,12 vezes não deve ser atribuído apenas ao Worker.

O resultado converge, na direção, com o de Dhaniswara et al. [5], que, no Laravel e em um *endpoint* leve, observaram no FrankenPHP em modo Worker vazão 18,6% maior, tempo médio 15,8% menor e mais memória (cerca de 155 MB contra 69 MB; aqui, +46 MiB). A magnitude, porém, difere (aqui, +112,1% de vazão e −57% de p95 sem saturação), e os estudos diferem em *framework*, *endpoints*, banco, ferramenta de carga, versão do PHP, recursos (4 núcleos contra 1,0 CPU) e na ausência do modo clássico, que aqui trouxe ganho pequeno; trata-se de convergência qualitativa, e não de replicação.

O experimento mede o efeito *combinado* do modo Worker: a eliminação do *bootstrap* a cada requisição e o reaproveitamento da conexão com o banco, que no CI4 em Worker também é preservada. Os dados por *endpoint* sugerem que, em latência, a parcela da conexão é a maior: o `/db` perde cerca de 11 ms, enquanto o `/health` perde menos de 0,5 ms. Isso é consistente com o custo de abrir e autenticar uma conexão PostgreSQL por requisição, mas é uma inferência, pois o `/health` não é um isolador perfeito do *bootstrap* e não houve perfilamento.

Do ponto de vista de eficiência de recursos, o C processou 2,1 vezes mais requisições que o A com a mesma CPU e apenas 46 MiB a mais de RAM, o que significa menos instâncias para a mesma carga. O cenário A usou menos CPU que os demais em modelo fechado (92,7%), possivelmente porque a cota é dividida entre dois contêineres (PHP-FPM e Nginx) e a capacidade ociosa de um não pode ser usada pelo outro; isso também faz parte do efeito medido entre A e as arquiteturas do FrankenPHP.

Esperava-se que o CI4, mais leve, superasse o Octane; ocorreu o contrário. O Octane apresentou vazão 11,0% superior à do CI4 Worker no teste fechado, diferença significativa (Welch com Holm) nas cinco execuções por cenário, e p95 cerca de 6% menor a 100 req/s, também significativo (Welch com Holm, $p = 0{,}009$) nas três execuções por cenário desse teste. Esses resultados se referem às implementações, à carga e às configurações avaliadas e não permitem concluir pela superioridade geral de um *framework*. O Octane também usou mais memória (172 contra 125 MiB). O Octane é um pacote maduro, projetado para esse modelo [13], enquanto o suporte do CI4 é recente [4]; essas explicações são hipóteses. A estabilidade da memória em 30 minutos reduz a preocupação com vazamento, mas não a elimina para períodos mais longos ou aplicações complexas.

5.1 Ameaças à validade

As execuções ocorreram em um único hospedeiro, com o injetor de carga na mesma máquina, e em dois dias; as duas últimas execuções de cada cenário foram coletadas no segundo dia. No primeiro dia, o cenário E foi executado em bloco, ao final, o que pode ter introduzido efeito de ordem ou de temperatura na comparação E com B; não houve tempo para repetir a coleta com ordem uniforme. As execuções de um mesmo cenário não são independentes no sentido estrito (mesmo *hardware*, dois dias, cenário E em bloco no primeiro), e o teste de Welch supõe independência; por isso os valores de p são condicionais a este ambiente e não demonstram generalização para outros. O IC descreve a variação entre execuções. Fatores que diferem entre os cenários e não são isolados: o PHP *thread-safe* do FrankenPHP contra o não *thread-safe* do PHP-FPM, a divisão da cota de CPU entre PHP-FPM e Nginx no cenário A, e a conexão persistente. A vazão foi medida em uma única carga fechada (100 usuários virtuais), sem varredura de concorrência, de modo que não se demonstra que seja o máximo de cada cenário (CPU próxima de 100% em B–E, mas 92,7% em A); a latência sem saturação foi medida em uma única taxa (100 req/s); a carga é sintética, com *endpoints* simples, e não exercita a condição observada por Dhaniswara et al. [5] em um *endpoint* que serializa grande volume de dados, em que a CPU saturou e o gargalo esteve na aplicação; as imagens de monitoramento não tiveram a versão fixada. Os resultados valem para este ambiente e não substituem testes com aplicações reais.

6 CONCLUSÕES E TRABALHOS FUTUROS

Neste experimento, o modo Worker do FrankenPHP elevou em 112,1% a vazão do CodeIgniter 4 sobre o PHP-FPM sob 100 usuários virtuais (304,4 contra 143,6 RPS; $n = 5$) e reduziu em 57% o p95 de uma carga abaixo da saturação (6,6 contra 15,3 ms), ao custo de cerca de 46 MiB de RAM, com memória estável em 30 minutos e preservação de estado e conexão equivalentes às do Laravel Octane, que ficou 11,0% acima em vazão. O ganho observado entre as arquiteturas completas é o efeito combinado da execução persistente, do reaproveitamento da conexão com o banco e de diferenças de servidor; contra o FrankenPHP clássico, a vantagem em vazão do Worker foi de 95,5%. Trocar apenas o servidor, mantendo o modo clássico, não se justificaria pelo desempenho. Para equipes com aplicações em CI4, os dados indicam que vale avaliar o modo Worker, observando a responsabilidade de evitar vazamento de estado entre requisições.

6.1 Trabalhos Futuros

Sugerem-se: variar a taxa de chegada para obter a curva de latência por carga; separar o custo do *bootstrap* do da conexão com perfilamento; testes de várias horas com aplicações mais realistas, com sessões, consultas complexas e *endpoints* com grande volume de dados; perfilamento para explicar a diferença entre C e D; e a comparação com Swoole e RoadRunner.

AGRADECIMENTOS

Agradecimentos omitidos para fins de avaliação cega.

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Foi utilizada a ferramenta de IA generativa Claude (Anthropic), por meio do Claude Code, como apoio na elaboração dos *scripts* de automação da coleta e da análise estatística, na organização das tabelas e figuras e na redação e revisão textual deste trabalho. O autor definiu o desenho experimental, executou a coleta, conferiu os valores apresentados com os arquivos de resultados gerados pelos *scripts*, revisou integralmente o conteúdo produzido e assume total responsabilidade pela precisão, originalidade e integridade científica do trabalho.

REFERENCES

[1] Anonymous. 2026. *cotb-benchmark-anon-F666: código-fonte, scripts e dados brutos do experimento* (README.md). Anonymous GitHub. https://anonymous.4open.science/r/cotb-benchmark-anon-F666/. Acesso em: 21 out. 2026.

[2] A. S. Azzahidi, B. Wijayanto, A. Darmawan. 2025. Performance Evaluation of Backend Frameworks for REST API: A Comparative Study of Spring Boot, Flask, Express.js, Laravel FrankenPHP, and Gin. *Jurnal Teknik Informatika (JUTIF)* 6, 4 (2025), 2405–2419. https://doi.org/10.52436/1.jutif.2025.6.4.4811

[3] H. Brunst et al. 2022. First Experiences in Performance Benchmarking with the New SPEChpc 2021 Suites. arXiv:2203.06751. https://arxiv.org/abs/2203.06751

[4] CodeIgniter Foundation. 2026. *CodeIgniter 4.7.0 Changelog* (lançada em 1 fev. 2026; suporte experimental ao modo Worker do FrankenPHP). In *CodeIgniter 4 User Guide*. https://codeigniter.com/user_guide/changelogs/v4.7.0.html. Acesso em: 19 out. 2026.

[5] A. Dhaniswara, T. F. Kusumasari, E. N. Alam. 2025. Performance Evaluation of Laravel Application Deployment on Docker. In *Proc. 2025 International Conference on Information Technology and Computing (ICITCOM)*. IEEE, 170–175. https://doi.org/10.1109/ICITCOM66635.2025.11265204

[6] K. Dunglas. s.d. *FrankenPHP: The Modern Application Server for PHP*. https://frankenphp.dev/docs/. Acesso em: 19 out. 2026.

[7] Grafana Labs. s.d. *k6 Documentation: Load Testing for Engineering Teams*. https://grafana.com/docs/k6/latest/using-k6/metrics/. Acesso em: 19 out. 2026.

[8] Grafana Labs. s.d. *k6 Documentation: Open and closed models*. https://grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed/. Acesso em: 19 out. 2026.

[9] Z. Jia et al. 2013. The Implications of Diverse Applications and Scalable Data Sets in Benchmarking Big Data Systems. arXiv:1307.7943. https://arxiv.org/abs/1307.7943

[10] C. Laaber, M. Basmaci, P. Salza. 2021. Predicting unstable software benchmarks using static source code features. *Empirical Software Engineering* 26, art. 114 (2021). https://doi.org/10.1007/s10664-021-09996-y

[11] J. D. C. Little. 1961. A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research* 9, 3 (1961), 383–387. https://doi.org/10.1287/opre.9.3.383

[12] J. M. da Luz. 2024. *Otimização de aplicações web com programação assíncrona em PHP e Swoole*. Trabalho de Conclusão de Curso (Tecnologia em Análise e Desenvolvimento de Sistemas), Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul, Campus Veranópolis. https://dspace.ifrs.edu.br/xmlui/handle/123456789/2099. Acesso em: 20 out. 2026.

[13] T. Otwell. s.d. *Laravel Octane*. Documentação do Laravel 13.x. Laravel LLC. https://laravel.com/docs/13.x/octane. Acesso em: 19 out. 2026.

[14] B. Schroeder, A. Wierman, M. Harchol-Balter. 2006. Open Versus Closed: A Cautionary Tale. In *Proc. 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*, San Jose, CA. https://static.usenix.org/events/nsdi06/tech/full_papers/schroeder/schroeder_html/index.html

[15] J. Zhan et al. 2022. A BenchCouncil View on Benchmarking Emerging and Future Computing. arXiv:2205.07769. https://arxiv.org/abs/2205.07769